Proyecto Personal

Administración de Bases de Datos en Producción

PostgreSQL · Linux · Automatización

Laboratorio de PostgreSQL en Producción

Alta disponibilidad, replicación, copias de seguridad y operación de un clúster PostgreSQL 16

Repositorio:
github.com/bogdanturcu13/postgres-production-lab

Fecha:
Octubre de 2026

Autor:
BOGDAN TURCU

Estudios:
Grado en Ingeniería Informática

Índice

1. Introducción y Objetivos **1**
1.1. Conceptos Básicos . . . 2

2. Arquitectura del Laboratorio **3**
2.1. Máquinas Virtuales . . . 4
2.2. Planificación por Fases . . . 4
2.3. Estructura del Repositorio . . . 5

3. Fase 0: Preparación del Entorno **5**
3.1. Equipo y Decisiones Previas . . . 6
3.1.1. Hipervisor . . . 6
3.1.2. Sistema Operativo . . . 6
3.1.3. Disco . . . 7
3.2. Repositorio y Control de Versiones . . . 7
3.2.1. Problema: Error 403 al Subir Cambios . . . 7
3.3. Configuración de Hyper-V . . . 8
3.4. Red Interna con NAT . . . 8
3.4.1. Problemas Encontrados . . . 9
3.5. Creación de las Máquinas Virtuales . . . 10
3.5.1. Decisiones de Diseño . . . 11
3.5.2. Verificación . . . 11
3.6. Instalación Manual de Rocky Linux en infra . . . 12
3.6.1. Acceso como Root . . . 12
3.6.2. Aviso de SMT . . . 13
3.6.3. SELinux . . . 13
3.6.4. Problema: Errores al Reiniciar tras la Instalación . . . 14
3.6.5. Verificación . . . 14
3.7. Acceso SSH por Clave . . . 14
3.8. Instalación Automática con Kickstart . . . 15
3.9. Actualización y Conexión entre Máquinas . . . 17
3.10. Resultado de la Fase . . . 18
3.11. Diferencias con un Entorno de Producción . . . 18

A. Scripts de Hyper-V **18**
A.1. 01-red.ps1 . . . 18
A.2. 01-red-borrar.ps1 . . . 19
A.3. 02-vms.ps1 . . . 19
A.4. 03-kickstart.ps1 . . . 20
A.5. 04-tras-kickstart.ps1 . . . 21

B. Registro de Decisiones **21**
B.1. ADR 0001: Hipervisor Hyper-V . . . 21
B.2. ADR 0002: Sistema Operativo Rocky Linux 9 . . . 22

1. Introducción y Objetivos

Este proyecto es un laboratorio personal para aprender a administrar PostgreSQL como se hace en un entorno de producción. La idea no es solo instalar una base de datos, sino montar un entorno parecido al de una empresa, operarlo día a día y romperlo a propósito para aprender a diagnosticar y arreglar los problemas, igual que lo haría un administrador de bases de datos (DBA) o un ingeniero de fiabilidad (SRE).

PostgreSQL es un sistema gestor de bases de datos relacional de código abierto. Funciona con un modelo cliente-servidor: el servidor guarda los datos y atiende las consultas SQL que le envían las aplicaciones (los clientes) por la red. Un entorno de **producción** es aquel en el que la base de datos da servicio real a usuarios o aplicaciones, de modo que cualquier caída o pérdida de datos tiene consecuencias. El **DBA** (*Database Administrator*) es quien se encarga de que ese servidor funcione, rinda y no pierda datos. El **SRE** (*Site Reliability Engineering*) es un enfoque de operación nacido en Google que aplica técnicas de ingeniería de software, sobre todo la automatización y la medición, a mantener los servicios fiables.

Administrar una base de datos en producción va mucho más allá de escribir consultas SQL. Hay que garantizar que el servicio siga disponible cuando falla un servidor, que los datos se puedan recuperar después de un error humano, que el rendimiento se mantenga a medida que crecen los datos y que cualquier cambio se aplique sin interrumpir el servicio. Son competencias difíciles de adquirir sin un entorno en el que practicar y, sobre todo, en el que equivocarse sin consecuencias.

Los objetivos del proyecto son los siguientes:

- **Administración:** Instalar, configurar y mantener PostgreSQL en producción.
- **Alta disponibilidad:** Montar un clúster con Patroni y etcd, y comprobar su comportamiento ante fallos reales.
- **Replicación:** Configurar replicación física (*streaming*) y lógica, y conocer sus limitaciones.
- **Copias de seguridad:** Hacer backups, restauraciones y recuperación ante desastres con pgBackRest, incluida la recuperación a un instante concreto (PITR) y la medición del RTO y el RPO.
- **Monitorización y rendimiento:** Monitorizar el clúster, analizar consultas lentas y planificar la capacidad.
- **Mantenimiento:** `VACUUM`, `ANALYZE`, `REINDEX`, control del *bloat* y ajuste de autovacuum.
- **Cambios en producción:** Aplicar parches y actualizar de versión (de la 16 a la 18) siguiendo un proceso formal de gestión de cambios.
- **Incidencias:** Resolver incidencias, analizar su causa raíz y documentarlas como postmortems siguiendo ITIL.
- **Automatización:** Automatizar todo con Bash y Ansible, hasta poder destruir el laboratorio y reconstruirlo desde cero.

El resultado final es un repositorio público en GitHub[1] con la infraestructura como código, los procedimientos de operación (*runbooks*), los informes de incidencias (postmortems) y la documentación del proyecto.

[1] https://github.com/bogdanturcu13/postgres-production-lab

1.1. Conceptos Básicos

Antes de entrar en el laboratorio conviene aclarar los conceptos que aparecen en los objetivos. Los más específicos de cada fase se explican en su apartado correspondiente.

Alta disponibilidad. Es la capacidad de un sistema de seguir dando servicio aunque falle alguno de sus componentes. En una base de datos se consigue con varias copias del servidor: un **primario**, que es el único que acepta escrituras, y una o varias **réplicas**, que mantienen una copia actualizada de los datos. Si el primario cae, una réplica se promociona a primario; a esto se le llama **failover** cuando ocurre por un fallo y **switchover** cuando se hace de forma planificada, por ejemplo para hacer un mantenimiento. Al conjunto de servidores que trabajan juntos se le llama **clúster**, y a cada uno de ellos, **nodo**.

WAL y replicación. PostgreSQL no escribe los cambios directamente en los ficheros de datos. Primero los anota en un registro secuencial llamado **WAL** (*Write-Ahead Log*) y solo después los aplica a los datos. Gracias a eso, si el servidor se apaga de golpe, al arrancar repasa el WAL y recupera los cambios que no le dio tiempo a aplicar. El WAL es también la base de casi todo lo demás: la **replicación física** consiste en enviar el WAL del primario a las réplicas para que apliquen exactamente los mismos cambios, y la **replicación lógica** envía en su lugar los cambios fila a fila, lo que permite copiar solo algunas tablas o hacia otra versión de PostgreSQL.

Copias de seguridad y recuperación. La alta disponibilidad no sustituye a las copias de seguridad: si alguien borra una tabla por error, el borrado se replica a todas las réplicas en milisegundos. Las copias de seguridad sirven para volver atrás. Si además se guarda todo el WAL generado desde la copia, se puede restaurar la base de datos a cualquier instante concreto, por ejemplo justo antes del borrado; es lo que se conoce como **PITR** (*Point-In-Time Recovery*). La recuperación ante desastres (DR) se mide con dos valores:

- **RTO** (*Recovery Time Objective*): cuánto tiempo se tarda en volver a dar servicio.
- **RPO** (*Recovery Point Objective*): cuántos datos se pierden, medido en tiempo (por ejemplo, los últimos 5 minutos).

Mantenimiento. En PostgreSQL, cuando una fila se modifica o se borra, la versión antigua no desaparece en el momento, sino que se queda ocupando espacio hasta que se limpia. `VACUUM` hace esa limpieza, `ANALYZE` actualiza las estadísticas que usa PostgreSQL para decidir cómo ejecutar cada consulta y `REINDEX` reconstruye los índices. El *bloat* es el espacio desperdiciado cuando esa limpieza no se hace a tiempo, y autovacuum es el proceso que la hace automáticamente.

Operación. Un **runbook** es un procedimiento paso a paso para una tarea concreta (por ejemplo, reconstruir una réplica), escrito para que cualquiera pueda seguirlo en mitad de una incidencia. Un **postmortem** es el informe que se escribe después de una incidencia para explicar qué pasó, por qué y qué se va a cambiar para que no se repita. **ITIL** es un conjunto de buenas prácticas de gestión de servicios de TI muy extendido en empresas, que distingue entre *incidentes* (algo ha fallado y hay que restablecer el servicio), *problemas* (la causa de fondo de uno o varios incidentes) y *cambios* (cualquier modificación planificada en producción). Por último, la **infraestructura como código** consiste en describir los servidores y su configuración en ficheros de texto versionados, de forma que se puedan reconstruir de manera automática y repetible; en este proyecto se hace con scripts y con Ansible, una herramienta que configura servidores remotos a través de SSH.

2. Arquitectura del Laboratorio

El laboratorio se ejecuta en un único portátil con Windows 11 Pro, sobre el hipervisor Hyper-V. Está formado por cuatro máquinas virtuales con Rocky Linux 9:

- `pg1`, `pg2` y `pg3`: Los tres nodos de base de datos. Uno hará de primario y los otros dos de réplicas.
- `infra`: La máquina de infraestructura, que concentra los servicios de apoyo.

La tabla 1 resume para qué sirve cada componente. Cada uno se instala y se explica en detalle en la fase en la que se usa.

Componente	**Máquina**	**Función**
PostgreSQL 16	`pg1`–`pg3`	El servidor de base de datos.
Patroni	`pg1`–`pg3`	Vigila PostgreSQL en cada nodo y, si cae el primario, promociona una réplica de forma automática.
etcd	`pg1`–`pg3`	Almacén de datos distribuido que usa Patroni para ponerse de acuerdo sobre qué nodo es el primario. Funciona por **quorum**: solo toma decisiones si responde la mayoría de sus nodos (2 de 3), lo que evita que dos nodos se crean primarios a la vez.
HAProxy	`infra`	Balanceador de carga: recibe las conexiones de los clientes y las envía al nodo que sea primario en cada momento.
PgBouncer	`infra`	*Pooler* de conexiones: mantiene abiertas unas pocas conexiones con PostgreSQL y las reparte entre muchos clientes, porque cada conexión directa a PostgreSQL consume bastante memoria.
pgBackRest	`infra`	Herramienta de copias de seguridad de PostgreSQL, con copias completas, diferenciales e incrementales y archivado del WAL.
MinIO	`infra`	Almacenamiento de objetos compatible con Amazon S3, donde pgBackRest guarda las copias.
Prometheus	`infra`	Recoge y guarda métricas (uso de CPU, conexiones, retraso de la replicación...) de todas las máquinas.
Alertmanager	`infra`	Envía avisos cuando una métrica supera un umbral.
Grafana	`infra`	Muestra las métricas en paneles gráficos.
Ansible	`infra`	Automatiza la instalación y la configuración del resto de máquinas.

Tabla 1: Componentes del laboratorio.

Las cuatro máquinas comparten una red interna aislada, 10.10.10.0/24, en la que el propio Windows hace de puerta de enlace y traduce las direcciones (NAT) hacia la red real. Estos conceptos de red se explican en la sección 3.4. La figura 1 resume la arquitectura.

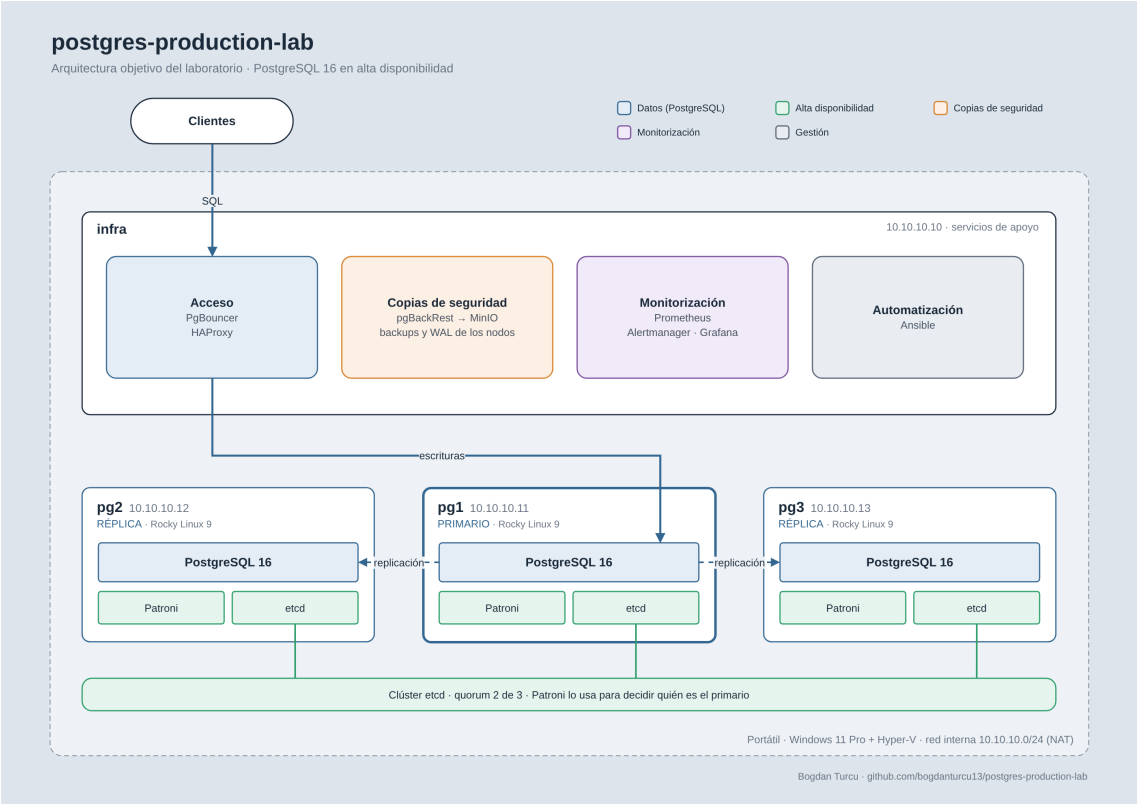


Figura 1: Arquitectura del laboratorio.

2.1. Máquinas Virtuales

Una **máquina virtual** (VM) es un ordenador simulado por software: tiene su propia CPU, memoria, discos y tarjeta de red virtuales, y en ella se instala un sistema operativo completo como si fuera un equipo físico. Varias máquinas virtuales pueden funcionar a la vez en un mismo equipo real, repartiéndose sus recursos. A cada procesador que se asigna a una máquina virtual se le llama **vCPU** (CPU virtual).

Todas las máquinas tienen un disco para el sistema operativo y otro distinto para los datos. Los motivos de esta separación se explican en la sección 3.5.

VM	IP	vCPU	RAM	Disco SO	Disco datos	Rol
`infra`	10.10.10.10	2	4 GB	20 GB	40 GB	Infraestructura
`pg1`	10.10.10.11	2	3 GB	20 GB	20 GB	Base de datos
`pg2`	10.10.10.12	2	3 GB	20 GB	20 GB	Base de datos
`pg3`	10.10.10.13	2	3 GB	20 GB	20 GB	Base de datos

Tabla 2: Inventario de máquinas virtuales.

2.2. Planificación por Fases

El trabajo se divide en diez fases. Cada una tiene sus objetivos, un ejercicio en el que se provoca un fallo a propósito y un entregable en el repositorio. No se pasa a la siguiente fase hasta cumplir el criterio de salida de la anterior. Los términos propios de cada fase que aparecen en la tabla se explican al llegar a ella.

Fase	Tema	Contenido
0	Preparación	Máquinas virtuales, red, acceso SSH por clave y repositorio.
1	Instalación	PostgreSQL 16 desde PGDG, memoria, logging, checkpoints, roles y PgBouncer.
2	Replicación	Streaming replication, slots, modo síncrono y asíncrono, replicación lógica.
3	Alta disponibilidad	Patroni, etcd y HAProxy. Switchover, failover y pérdida de quorum.
4	Backups y DR	pgBackRest con MinIO, PITR, restauración completa, RTO y RPO.
5	Monitorización	Prometheus, Grafana, alertas, análisis de consultas y capacidad.
6	Mantenimiento	MVCC, *bloat*, autovacuum, `REINDEX` y *wraparound*.
7	Cambios	Parches con reinicio escalonado y actualización de la versión 16 a la 18.
8	Incidencias e ITIL	Incidentes, problemas y cambios en ServiceNow, guardias y postmortems.
9	Automatización	Roles de Ansible y reconstrucción completa del laboratorio.

Tabla 3: Fases del proyecto.

2.3. Estructura del Repositorio

Todo lo que se hace en el laboratorio queda documentado o automatizado en el repositorio:

Ruta	Contenido
`docs/apuntes.md`	Cuaderno de trabajo con lo hecho en cada bloque, los problemas encontrados y su solución. Es la base de este documento.
`docs/decisiones/`	Registro de decisiones de arquitectura (ADR, *Architecture Decision Record*): documentos breves que explican por qué se ha elegido cada cosa frente a sus alternativas.
`docs/preguntas.md`	Preguntas de repaso tipo entrevista técnica de cada fase.
`docs/plantillas/`	Plantillas de runbook, postmortem y ADR.
`scripts/`	Scripts para crear y preparar el laboratorio.
`ansible/`	Automatización del despliegue (a partir de la Fase 1).
`runbooks/`, `postmortems/`	Procedimientos de operación e informes de incidencias.

Tabla 4: Estructura del repositorio.

Cada tarea sigue el mismo método: primero se explica el concepto, después se ejecutan los pasos y por último se comprueba que funciona con comandos de verificación. Cuando algo falla, se diagnostica antes de aplicar una solución.

3. Fase 0: Preparación del Entorno

En esta primera fase se prepara la base sobre la que se construye el resto del proyecto: el hipervisor, la red, las cuatro máquinas virtuales con su sistema operativo, el acceso remoto seguro y el repositorio. El criterio de salida es tener acceso SSH por clave entre todas las máquinas y el repositorio creado con su estructura.

3.1. Equipo y Decisiones Previas

El laboratorio se ejecuta en el siguiente equipo:

Componente	Detalle
Procesador	Intel Core i5-11400H, 6 núcleos y 12 hilos
Memoria	32 GB
Sistema operativo	Windows 11 Pro
Disco del laboratorio	Unidad E:, NVMe Crucial P2 (CT1000P2SSD8), con 484 GB libres

Tabla 5: Características del equipo.

El procesador tiene 6 núcleos físicos, pero cada uno puede ejecutar dos hilos a la vez gracias a una técnica llamada SMT (*Simultaneous Multithreading*, o *Hyper-Threading* en Intel), por eso el sistema ve 12. Con estos recursos cabe la arquitectura completa sin recortes: las cuatro máquinas suman 8 vCPU y 13 GB de RAM, y a Windows le quedan casi 19 GB.

3.1.1. Hipervisor

Para crear máquinas virtuales hace falta un **hipervisor**, que es el software que reparte el hardware real entre ellas y las mantiene aisladas unas de otras. Hay dos tipos:

- **Tipo 1 (*bare metal*):** Se ejecuta directamente sobre el hardware, sin un sistema operativo por debajo. Es el que se usa en los centros de datos. Ejemplos: Hyper-V, VMware ESXi, KVM.
- **Tipo 2:** Es una aplicación más que se ejecuta dentro de un sistema operativo. Es más sencillo de usar, pero añade una capa intermedia. Ejemplos: VirtualBox, VMware Workstation.

Se ha elegido **Hyper-V**, el hipervisor de Microsoft, que viene incluido en Windows 11 Pro. Aunque se activa desde Windows, es de tipo 1: al activarlo, lo primero que se carga al arrancar el equipo es el hipervisor, y Windows se ejecuta encima de él en la *partición raíz*, una máquina virtual con privilegios especiales que gestiona los controladores y crea el resto de máquinas. Por eso parece que Windows sigue siendo el sistema principal, aunque en realidad no lo es.

Además, Hyper-V no choca con WSL2 (el subsistema de Windows para ejecutar Linux) ni con las funciones de seguridad de Windows 11 que se basan en la virtualización, como la integridad de memoria, que de hecho ya usan el hipervisor por debajo. VirtualBox, en cambio, tendría que funcionar encima de esa capa de Hyper-V y perdería bastante rendimiento de disco. Por último, Hyper-V se puede manejar entero con **PowerShell**, la consola y lenguaje de scripts de Windows, lo que permite automatizar la creación del laboratorio. Sus comandos se llaman *cmdlets* y siguen el formato *Verbo-Sustantivo*, como `New-VM` (crear una máquina) o `Get-VM` (consultarlas).

3.1.2. Sistema Operativo

Las máquinas usan **Rocky Linux 9**. Rocky es una copia gratuita y compatible 1:1 con Red Hat Enterprise Linux (RHEL), la distribución de Linux más habitual en grandes empresas y en banca: se compila a partir del mismo código fuente, así que los paquetes, las rutas y el comportamiento son los mismos. La diferencia con RHEL es que no requiere pagar una suscripción. Se eligió frente a Debian o Ubuntu porque obliga a trabajar con

dos piezas de seguridad que están activadas por defecto en ese tipo de entornos: **SELinux**, que se explica en la sección 3.6.3, y **firewalld**, el cortafuegos de la familia Red Hat, que organiza las reglas por servicios (por ejemplo, `ssh`) en lugar de por puertos sueltos.

Las dos decisiones se documentan como ADR en el repositorio (apéndice B).

3.1.3. Disco

Hay un detalle del disco que conviene tener presente. La unidad E: es un SSD NVMe (un SSD conectado directamente al bus PCI Express, mucho más rápido que los antiguos SATA) sin memoria DRAM propia. Los SSD de este tipo escriben primero en una pequeña zona rápida que hace de caché y, cuando se llena en escrituras largas y sostenidas, la velocidad cae bastante. Como las cuatro máquinas escriben en el mismo disco físico, habrá que tenerlo en cuenta al interpretar las métricas de disco en la Fase 5.

3.2. Repositorio y Control de Versiones

El proyecto se guarda en un repositorio de **Git**, un sistema de control de versiones que registra cada cambio en los ficheros como un *commit*, con su autor, su fecha y un mensaje que lo describe. El repositorio se sube (`git push`) a GitHub, donde es público desde el primer día. Por eso se crea con dos ficheros de configuración:

- `.gitignore`**:** Lista los ficheros que Git nunca debe subir: los discos virtuales, las ISO, las claves privadas y cualquier fichero de secretos.
- `.gitattributes`**:** Fuerza los finales de línea de tipo LF. Windows termina cada línea de un fichero de texto con dos caracteres invisibles, CR y LF (*CRLF*), mientras que Linux solo usa LF. Si un script editado en Windows llega a Linux con CRLF, el carácter CR sobrante hace que falle. La excepción son los scripts de PowerShell, que se dejan en CRLF porque se ejecutan en Windows.

3.2.1. Problema: Error 403 al Subir Cambios

En Git hay que distinguir dos cosas que no tienen nada que ver: la **identidad** con la que se firman los commits (un nombre y un correo, configurados con `user.name` y `user.email`) y la **cuenta** con la que uno se autentica en GitHub al subir los cambios. El problema vino de la segunda.

- **Síntoma:** `git push` devolvía *Permission denied to ualbt635*. Git se autenticaba con la cuenta de GitHub de la universidad en lugar de con la personal.
- **Causa:** Al conectarse por HTTPS, Git usa las credenciales guardadas en el Administrador de credenciales de Windows. Ahí estaban las de la otra cuenta, y Git las reutilizaba sin preguntar.
- **Solución:** Se cambió HTTPS por SSH, con una clave y un alias por cuenta (`github-personal` y `github-uni`) en `~/.ssh/config`. Así, la cuenta queda escrita en la propia dirección del repositorio, por ejemplo `git@github-personal:bogdanturcu13/...`. La identidad de los commits se elige según la carpeta en la que está el repositorio con la directiva `includeIf` de la configuración de Git, y la opción `user.useConfigOnly` impide hacer un commit si no hay ninguna identidad configurada, en lugar de inventarse una.

3.3. Configuración de Hyper-V

Por defecto, Hyper-V guarda las máquinas y sus discos en la unidad C:, que solo tenía 14,5 GB libres. Antes de crear nada se movieron las rutas a la unidad E: desde PowerShell como administrador:

```
Enable-WindowsOptionalFeature -Online -FeatureName Microsoft-Hyper-V -All

New-Item -ItemType Directory -Path E:\HyperV\VHD, E:\HyperV\VMs, E:\HyperV\ISO
     -Force
Set-VMHost -VirtualHardDiskPath 'E:\HyperV\VHD' -VirtualMachinePath 'E:\HyperV
    \VMs'
Get-VMHost | Select-Object VirtualHardDiskPath, VirtualMachinePath
```

El primer comando activa Hyper-V; `New-Item` crea las carpetas y `Set-VMHost` cambia las rutas por defecto de los discos virtuales (ficheros `.vhdx`) y de la configuración de las máquinas. En este equipo Hyper-V ya estaba activado: el primer comando devolvió `RestartNeeded : False` y la propiedad `HypervisorPresent` de `Win32_ComputerSystem` valía `True`, así que no hizo falta reiniciar.

Por último, se descargó la imagen de instalación `Rocky-9.8-x86_64-minimal.iso`. Una imagen ISO es un fichero que contiene el contenido completo de un disco óptico, en este caso el instalador del sistema. La versión *minimal* incluye solo lo imprescindible para instalar un servidor sin entorno gráfico.

3.4. Red Interna con NAT

Antes de ver cómo se ha montado la red, conviene repasar los conceptos que intervienen:

- **Dirección IP y máscara:** Cada equipo de la red tiene una dirección IP, como 10.10.10.11. La notación `10.10.10.0/24` indica una red cuyos primeros 24 bits (los tres primeros números) son fijos, de modo que admite equipos desde 10.10.10.1 hasta 10.10.10.254. Es lo mismo que la máscara 255.255.255.0.
- **Puerta de enlace:** Es el equipo al que una máquina envía todo el tráfico dirigido a otras redes, por ejemplo a Internet. Hace de router.
- **NAT** (*Network Address Translation*): Las direcciones 10.x.x.x son privadas y no se pueden usar en Internet. Con NAT, el router sustituye la dirección privada de origen de cada paquete por su propia dirección real y, cuando llega la respuesta, deshace el cambio para entregarla a la máquina correcta. Es lo mismo que hace el router de casa.
- **DHCP:** Protocolo que asigna direcciones IP automáticamente a los equipos que se conectan a una red.
- **DNS:** Sistema que traduce nombres como `google.com` a direcciones IP. En el laboratorio se usan los servidores DNS públicos 1.1.1.1 (Cloudflare) y 8.8.8.8 (Google).
- **Switch virtual:** Igual que un switch físico conecta los equipos de una red local, un switch virtual conecta entre sí las tarjetas de red de las máquinas virtuales.

Las máquinas se conectan a un switch virtual **interno**, `LabSwitch`. Al crearlo, Windows recibe una tarjeta de red virtual conectada a él, `vEthernet (LabSwitch)`, en la que se le asigna la dirección 10.10.10.1. Windows hace así de puerta de enlace y de router con NAT hacia la red real. Con este diseño:

- **Direcciones fijas:** Las IP de las máquinas no dependen de la red a la que esté conectado el portátil.
- **Aislamiento:** Las máquinas salen a Internet para instalar paquetes, pero nadie de fuera puede entrar en ellas.
- **Sin DHCP:** Cada máquina tiene su dirección configurada a mano, como en producción, donde los servidores tienen direcciones fijas para que los clientes sepan siempre dónde encontrarlos.

La alternativa sería un switch **externo**, que conecta las máquinas directamente a la red real como si fueran equipos enchufados al router. En ese caso recibirían la dirección del router de casa por DHCP, cambiarían al cambiar de red y quedarían expuestas al resto de dispositivos.

La red se crea con el script `01-red.ps1` (apéndice A), que tiene solo tres comandos:

Comando	Función
`New-VMSwitch`	Crea el switch interno `LabSwitch`.
`New-NetIPAddress`	Asigna a Windows la IP 10.10.10.1/24 en ese switch.
`New-NetNat`	Crea el NAT `LabNAT` para la red 10.10.10.0/24.

Tabla 6: Comandos del script de red.

Para comprobar que funciona se usa `Get-NetNat`, que muestra el NAT activo, y `Test-Connection 10.10.10.1`, que hace *ping* a la nueva dirección (envía un pequeño paquete y espera respuesta, para comprobar que un equipo es alcanzable). En PowerShell 5.1, la columna `IPV4Address` de `Test-Connection` muestra la IP de **origen** (la de la red real), no la de destino. Lo importante es que hay respuesta.

```
Name                             : LabNAT
InternalIPInterfaceAddressPrefix : 10.10.10.0/24
Active                           : True
```

3.4.1. Problemas Encontrados

- **Fallo al crear el switch:** `New-VMSwitch` falló con el código `0x800700B7` (*No se puede crear un archivo que ya existe*) sin que existiera ningún switch ni adaptador con ese nombre. Como consecuencia también falló `New-NetIPAddress`, porque no existía la tarjeta `vEthernet (LabSwitch)`; el NAT sí se creó. Al repetir el comando, el switch se creó bien: fue un error puntual al registrar el adaptador virtual. Los posibles factores son restos de un adaptador de Hyper-V antiguo y controladores de VPN (Fortinet, Hamachi) en estado de error, que instalan filtros sobre todas las tarjetas de red del equipo.
- **Switch duplicado:** Al reintentar se crearon dos switches llamados `LabSwitch`. Hyper-V lo permite porque identifica cada switch internamente por un Id único y no por su nombre. Se solucionó borrando ambos con `Remove-VMSwitch -Name LabSwitch -Force` (script `01-red-borrar.ps1`), creando uno solo y asignando después la IP.

La lección es que los scripts sencillos no son **idempotentes**. Un script idempotente es el que se puede ejecutar varias veces y deja siempre el mismo resultado, porque antes de crear algo comprueba si ya existe. Como estos no lo son, antes de repetir un comando

hay que comprobar el estado con `Get-VMSwitch`, `Get-NetNat` y `Get-NetIPAddress`. Más adelante, Ansible sí será idempotente.

3.5. Creación de las Máquinas Virtuales

Antes de crear las máquinas conviene explicar algunos conceptos de su configuración:

- **Firmware, BIOS y UEFI:** El firmware es el pequeño programa que arranca un ordenador antes que el sistema operativo y le pasa el control. La BIOS es el firmware clásico; UEFI es su sustituto moderno, con más capacidades y con soporte para arranque seguro. Hyper-V ofrece máquinas de **generación 1**, que emulan una BIOS antigua, y de **generación 2**, que usan UEFI como cualquier servidor actual.
- **Arranque seguro** (*Secure Boot*): Función de UEFI que solo deja arrancar cargadores del sistema operativo firmados digitalmente con un certificado de confianza. Así se impide que un programa malicioso se cargue antes que el sistema. Hyper-V tiene varias plantillas con distintos certificados de confianza.
- **Disco virtual** (VHDX): Es un fichero en el disco real que la máquina virtual ve como un disco completo. Puede ser de **tamaño fijo**, que ocupa desde el principio todo su tamaño, o **dinámico**, que empieza casi vacío y crece a medida que se escribe en él.
- **Memoria dinámica:** Función de Hyper-V que da y quita RAM a las máquinas en caliente según la necesiten, mediante una técnica llamada *ballooning*.
- **Checkpoints:** Instantáneas del estado completo de una máquina en un momento dado, a las que se puede volver después. No hay que confundirlos con los *checkpoints* de PostgreSQL, que son otra cosa y se verán en la Fase 1.

Las cuatro máquinas se crean con el script `02-vms.ps1`, que recorre los cuatro nombres y crea todas iguales. Al final, a `infra` se le da algo más de RAM y de disco de datos.

Comando	Función
`New-VM`	Crea la máquina de generación 2 (UEFI), la conecta a `LabSwitch` y crea su disco de sistema.
`Set-VM`	Asigna 2 vCPU, fija la memoria y desactiva los checkpoints automáticos.
`New-VHD`	Crea el disco de datos.
`Add-VMHardDiskDrive`	Conecta el disco de datos a la máquina.
`Add-VMDvdDrive`	Añade un lector de DVD virtual con la ISO de Rocky. Equivale a meter un USB de instalación.
`Set-VMFirmware`	Configura el firmware UEFI de la máquina: la plantilla de arranque seguro y el arranque desde el DVD.
`Resize-VHD`	Amplía a 40 GB el disco de datos de `infra`.

Tabla 7: Comandos del script de creación de máquinas.

En los comandos, el primer valor sin nombre de parámetro (por ejemplo, `$n`) es un **parámetro posicional**: PowerShell lo asigna al parámetro principal, que en este caso es el nombre de la máquina (`-VMName`). Hyper-V lleva un inventario de las máquinas registradas, así que basta con el nombre para encontrarlas, sin indicar ninguna ruta.

3.5.1. Decisiones de Diseño

- **Disco de datos aparte:** Si los datos o el WAL llenan el disco (lo que se provocará a propósito en la Fase 2), PostgreSQL se para, pero el sistema operativo sigue funcionando y se puede entrar por SSH a arreglarlo. Además, el disco de datos puede crecer por separado y conservarse aunque se reinstale el sistema.
- **Memoria fija:** PostgreSQL reserva al arrancar una zona de memoria propia para guardar los datos más usados (`shared_buffers`), y su planificador de consultas da por hecho que dispone de cierta caché adicional (`effective_cache_size`). Esa caché adicional es la **caché de disco** de Linux, que usa la RAM libre para guardar los ficheros leídos recientemente y no tener que volver a leerlos del disco. Si el hipervisor le quita memoria a la máquina en caliente, lo primero que se pierde es esa caché y el rendimiento se vuelve impredecible. En el peor caso, el sistema empieza a usar la **swap** (una zona del disco que hace de memoria de reserva, mucho más lenta que la RAM) o el **OOM killer** (el mecanismo de Linux que, cuando se queda sin memoria, mata procesos para liberarla) termina con PostgreSQL.
- **Plantilla de arranque seguro:** La plantilla por defecto solo confía en los certificados de Windows. La plantilla `MicrosoftUEFICertificateAuthority` confía también en el certificado con el que está firmado `shim`, el primer cargador que arranca en las distribuciones Linux y que después carga el sistema.
- **Sin checkpoints automáticos:** Windows 11 los activa por defecto y crean instantáneas sin avisar que ocupan disco. Además, un checkpoint no es una copia de seguridad: está en el mismo disco que la máquina y, en una base de datos en marcha, equivale a un corte de luz.
- **Más recursos para `infra`:** Lleva más servicios (MinIO, Prometheus, Grafana) y guarda los backups y las métricas.

3.5.2. Verificación

```
Get-VM | Select-Object Name, ProcessorCount, @{n='RAM_GB';e={$_.MemoryStartup
    /1GB}}, DynamicMemoryEnabled
```

```
Name  ProcessorCount RAM_GB DynamicMemoryEnabled
----  -------------- ------ -------------------
infra              2      4               False
pg1                2      3               False
pg2                2      3               False
pg3                2      3               False
```

Con `Get-VMHardDiskDrive -VMName * | Get-VHD` se comprobaron los ocho discos con su tamaño máximo, y con `Get-VMFirmware` la plantilla de arranque seguro de las cuatro máquinas. Los discos son dinámicos: aunque su tamaño máximo es de 20 GB, recién creados ocupan solo 4 MB en E:.

Los discos dinámicos ahorran espacio, pero tienen inconvenientes en una base de datos. Si la unidad E: se llenara, Hyper-V pausaría todas las máquinas a la vez; cada vez que se escribe en una zona nueva el fichero tiene que crecer, lo que añade latencia; y al borrar datos el fichero no se reduce solo (hay que hacerlo a mano con `Optimize-VHD`). Por eso, en producción las bases de datos suelen usar discos de tamaño fijo.

3.6. Instalación Manual de Rocky Linux en infra

La primera máquina se instaló a mano, para entender cada decisión antes de automatizarla. La máquina se enciende con `Start-VM infra` y su pantalla se abre con `vmconnect localhost infra`, que es la consola de Hyper-V, el equivalente a conectarle un monitor y un teclado. En el instalador se eligieron estas opciones:

Apartado	Valor
Idioma	Inglés (Estados Unidos). En servidores es lo habitual, porque los mensajes de error se buscan mejor.
Teclado	Español.
Destino	Solo el disco de sistema (`sda`), con particionado automático. El de datos (`sdb`) queda sin tocar.
Red	Nombre `infra`, IP 10.10.10.10/24, puerta de enlace 10.10.10.1 y DNS 1.1.1.1 y 8.8.8.8.
Hora	Europe/Madrid, con la hora de red activada.
Software	Minimal Install.
Root	Con contraseña, pero sin permitir la entrada por SSH.
Usuario	`bogdan`, administrador.

Tabla 8: Opciones del instalador de Rocky Linux.

En Linux, los discos aparecen como ficheros dentro de `/dev`: el primero es `sda`, el segundo `sdb`, y así sucesivamente. El particionado automático crea en `sda` tres zonas:

- `/boot/efi`**:** Una pequeña partición que lee el firmware UEFI para encontrar el cargador del sistema.
- `/boot`**:** Guarda el kernel (el núcleo del sistema operativo) y los ficheros necesarios para arrancarlo.
- **Un volumen LVM:** El resto del disco. LVM (*Logical Volume Manager*) es una capa que agrupa el espacio de uno o varios discos y lo reparte en volúmenes lógicos que se pueden ampliar después sin reinstalar. Aquí contiene dos volúmenes: la raíz del sistema (`/`) y la swap.

3.6.1. Acceso como Root

En Linux, **root** es el usuario administrador, que puede hacer cualquier cosa en el sistema. El trabajo diario, sin embargo, se hace con un usuario normal y el comando `sudo`, que permite ejecutar una orden concreta con privilegios de root después de introducir la contraseña del propio usuario. En Rocky, pueden usar `sudo` los usuarios del grupo `wheel`, que es en el que se mete al usuario al marcarlo como administrador en el instalador. Se hace así por dos motivos:

- **Seguridad:** `root` es el primer usuario que prueban los ataques de fuerza bruta. Si no se puede entrar como root, el atacante tiene que adivinar también el nombre del usuario.
- **Trazabilidad:** Cada `sudo` queda registrado con el usuario que lo ejecutó (en Rocky Linux, en `/var/log/secure`). Si todos entran como root, no hay forma de saber quién hizo qué.

Se descartó bloquear la cuenta de root por completo. Eso no impediría usar `sudo`, pero sí entrar en el **modo de emergencia**, un modo de arranque mínimo al que entra el sistema

cuando algo impide arrancar con normalidad y que pide la contraseña de root. Si la cuenta estuviera bloqueada y el sistema no arrancara, habría que recuperarlo desde la ISO. Con root con contraseña pero sin SSH se tienen las dos ventajas.

3.6.2. Aviso de SMT

Durante la instalación apareció un aviso sobre SMT. Al compartir dos hilos los recursos de un mismo núcleo, un proceso puede llegar a deducir datos de otro que se esté ejecutando en el mismo núcleo, aprovechando fallos de diseño de los procesadores como Spectre o MDS. Red Hat avisa por si se quiere desactivar SMT, a costa de perder rendimiento. No se desactivó: las máquinas solo ejecutan código propio, el kernel ya trae activadas las mitigaciones de esas vulnerabilidades y Hyper-V aísla las máquinas entre sí. Esta medida solo tiene sentido en entornos donde varios clientes desconocidos comparten el mismo hardware, como las nubes públicas.

3.6.3. SELinux

En Linux, los permisos de toda la vida (lectura, escritura y ejecución para el dueño, el grupo y el resto) los decide el propietario de cada fichero; es lo que se llama **control de acceso discrecional** (DAC). El problema es que, si un atacante toma el control de un servicio, puede hacer todo lo que permitan los permisos del usuario con el que se ejecuta ese servicio.

SELinux (*Security-Enhanced Linux*) añade por encima una segunda capa de **control de acceso obligatorio** (MAC): unas reglas que fija el sistema y que ningún usuario, ni siquiera el propietario del fichero, puede saltarse. Para ello pone una **etiqueta** (o *contexto*) a cada proceso y a cada fichero. La parte importante de la etiqueta es el **tipo**, que suele terminar en `_t`: por ejemplo, el servicio SSH se ejecuta con el tipo `sshd_t` y la carpeta `~/.ssh` lleva el tipo `ssh_home_t`. Las etiquetas se pueden ver añadiendo la opción `-Z` a comandos como `ls` o `ps`.

La **política** de SELinux es el conjunto de reglas que dice qué tipo de proceso puede hacer qué operación sobre qué tipo de fichero. Todo lo que la política no permite expresamente, se deniega. Rocky usa por defecto la política ***targeted***, que confina los servicios de red y los procesos de mayor riesgo (`sshd`, servidores web, bases de datos...) y deja sin restricciones al resto, como los programas del propio usuario. Así, aunque un atacante tomara el control de un servicio, solo podría tocar los ficheros con las etiquetas que la política permite a ese servicio.

SELinux tiene tres modos, que se consultan con `getenforce`:

- ***Enforcing*:** Aplica las reglas y deniega lo que no está permitido. Es el modo por defecto y el que se usa en el laboratorio.
- ***Permissive*:** No deniega nada, pero registra lo que habría denegado. Sirve para diagnosticar problemas.
- ***Disabled*:** Desactivado. Es la "solución" que se ve en muchos tutoriales cuando algo no funciona y una mala práctica en producción. En este proyecto SELinux no se desactiva para resolver problemas.

Cada denegación queda registrada en `/var/log/audit/audit.log` como un mensaje `avc: denied`, que indica la operación, el tipo del proceso y el tipo del fichero. Es el primer sitio que hay que mirar cuando un servicio no puede acceder a un fichero aunque sus permisos sean correctos.

Cuando un paquete se instala con `dnf`, sus ficheros reciben automáticamente la etiqueta correcta. El problema aparece al crear o copiar ficheros a mano: pueden quedarse con una

etiqueta genérica, normalmente la de la carpeta en la que se crean, que no es la que espera el servicio. Para eso existe `restorecon`, que restaura en una ruta la etiqueta que la política le asigna, con `-R` para recorrer las subcarpetas y `-v` para mostrar los cambios. Si se quiere que una ruta nueva tenga una etiqueta distinta de la que le corresponde por defecto, primero se añade la regla a la política con `semanage fcontext` y después se aplica con `restorecon`. Se usará así en la sección 3.7 y en las fases siguientes.

3.6.4. Problema: Errores al Reiniciar tras la Instalación

- **Síntoma:** Al reiniciar tras la instalación aparecieron muchos errores (`I/O error, dev sr0`, `SQUASHFS error`, `Failed to execute shutdown binary`) y la máquina se quedó colgada.
- **Causa:** La ISO se quitó del lector (`sr0`, el lector de DVD virtual) mientras el instalador se estaba apagando. El instalador no se instala en el disco, sino que se ejecuta directamente desde la ISO, que guarda sus programas en un sistema de archivos comprimido de solo lectura llamado SquashFS. Al quitar la ISO, el instalador no pudo leer sus propios ficheros para terminar de apagarse. La instalación en sí ya había terminado bien.
- **Solución:** Comprobar que el lector estaba vacío, apagar la máquina con `Stop-VM infra -TurnOff` (que equivale a quitarle la corriente) y volver a encenderla. En las siguientes máquinas, la ISO se quita después de que se reinicien.

3.6.5. Verificación

- `ip -br a`**:** Muestra las tarjetas de red en formato breve. La tarjeta `eth0` está activa con la dirección 10.10.10.10/24.
- `ping google.com`**:** Responde, lo que confirma a la vez la puerta de enlace, el NAT y el DNS.
- `lsblk`**:** Lista los discos y sus particiones. `sda` tiene el sistema (`/boot/efi`, `/boot` y el volumen LVM con la raíz y 2 GB de swap) y `sdb` está vacío.

En servidores de bases de datos la swap se deja pequeña, porque si PostgreSQL empieza a usarla el rendimiento se hunde. Se revisará en la Fase 1.

3.7. Acceso SSH por Clave

SSH (*Secure Shell*) es el protocolo que se usa para conectarse a un servidor remoto y manejarlo desde la consola, con todo el tráfico cifrado. Tiene dos partes: el servidor, `sshd`, que se ejecuta en cada máquina y escucha en el puerto 22, y el cliente, `ssh`, que se ejecuta en el equipo desde el que uno se conecta.

En lugar de entrar con contraseña se usa **autenticación por clave pública**, basada en criptografía asimétrica. Se genera un par de claves relacionadas matemáticamente, de forma que lo que se firma con una solo se puede verificar con la otra, pero a partir de la pública no se puede obtener la privada:

- **Clave privada:** Se queda en el equipo del administrador y nunca sale de ahí.
- **Clave pública:** Se copia a cada servidor, en el fichero `~/.ssh/authorized_keys` del usuario, que es la lista de claves autorizadas para entrar con ese usuario.

Al conectar, el cliente firma unos datos de la sesión con su clave privada y el servidor comprueba la firma con la pública. Si es válida, deja entrar sin pedir contraseña. Es más seguro, porque no hay ninguna contraseña que adivinar, y además es lo que necesita Ansible para conectarse solo a las máquinas. Las claves son de tipo **Ed25519**, que usa el algoritmo de firma EdDSA sobre curvas elípticas; es el estándar actual, más rápido y con claves más cortas que RSA. Se ha creado un par exclusivo para el laboratorio:

```
ssh-keygen -t ed25519 -f $HOME\.ssh\id_ed25519_lab -C "bogdan@lab"
type $HOME\.ssh\id_ed25519_lab.pub | ssh bogdan@10.10.10.10 "mkdir -p ~/.ssh
    && chmod 700 ~/.ssh && cat >> ~/.ssh/authorized_keys && chmod 600 ~/.ssh/
    authorized_keys && restorecon -R ~/.ssh"
```

El segundo comando lee la clave pública en Windows y se la pasa por SSH a la máquina, que crea la carpeta `~/.ssh` y añade la clave a `authorized_keys`. Los permisos 700 y 600 hacen que solo el propio usuario pueda acceder a la carpeta y al fichero; no es un detalle menor, porque `sshd` ignora las claves si cualquier otro usuario puede modificarlas. Por último, como la carpeta se ha creado a mano, `restorecon` le pone la etiqueta de SELinux que le corresponde, `ssh_home_t`. Sin ella, la carpeta podría quedarse con la etiqueta genérica de la carpeta personal, y `sshd`, que está confinado por SELinux, no podría leer las claves aunque los permisos fueran correctos: seguiría pidiendo contraseña.

Para no tener que escribir las IP se definen alias en `~/.ssh/config`, el fichero de configuración del cliente SSH. El último bloque aplica el mismo usuario y la misma clave a las cuatro máquinas:

```
Host infra
    HostName 10.10.10.10
Host pg1
    HostName 10.10.10.11
Host pg2
    HostName 10.10.10.12
Host pg3
    HostName 10.10.10.13

Host infra pg1 pg2 pg3
    User bogdan
    IdentityFile ~/.ssh/id_ed25519_lab
```

Igual que el cliente demuestra su identidad con su clave, el servidor también tiene su propio par de claves, que se genera al instalarlo. La primera vez que se entra en un servidor, SSH muestra la **huella** de su clave pública (un resumen SHA-256 de ella) y pide confirmar que es la correcta. La huella se guarda en el fichero `known_hosts` y, si en el futuro cambia, SSH avisa. Eso permite detectar un ataque de **intermediario** (*man-in-the-middle*), en el que alguien se hace pasar por el servidor para interceptar la conexión. La primera conexión funciona por confianza (lo que se llama TOFU, *Trust On First Use*), así que lo correcto es comprobar esa primera huella por otro canal, por ejemplo desde la consola del propio servidor.

3.8. Instalación Automática con Kickstart

Las tres máquinas de base de datos se instalaron de forma automática con **Kickstart**, que es la forma estándar de instalar en serie servidores de la familia Red Hat, tanto físicos como virtuales. El instalador de estas distribuciones se llama Anaconda, y Kickstart es un fichero de texto, `ks.cfg`, con todas las respuestas que normalmente se le dan a mano: idioma, discos, red, usuarios, paquetes... Si Anaconda lo encuentra, instala sin preguntar nada.

Para que lo encuentre sin tocar el menú de arranque, el fichero se mete en un disco pequeño cuyo sistema de archivos tiene la **etiqueta** `OEMDRV`. Una etiqueta es un nombre que se guarda dentro del propio sistema de archivos y que no depende del orden en que el sistema detecte los discos. Al arrancar el instalador, antes de que aparezca ninguna pantalla, se carga un pequeño sistema temporal llamado *initramfs*, que se encarga de detectar todos los discos; si encuentra uno con la etiqueta `OEMDRV`, lo monta y busca `ks.cfg` en su raíz. Ese disco se formatea en FAT32, un sistema de archivos sencillo que pueden leer y escribir tanto Windows como Linux. El contenido del fichero es el siguiente:

Listado 1: Plantilla del fichero Kickstart que genera el script.

```
text
lang en_US.UTF-8
keyboard es
timezone Europe/Madrid --utc
network --bootproto=static --device=eth0 --ip=<IP> --netmask=255.255.255.0 --gateway
    =10.10.10.1 --nameserver=1.1.1.1,8.8.8.8 --hostname=<nombre> --activate
ignoredisk --only-use=sda
clearpart --all --initlabel --drives=sda
autopart
bootloader --boot-drive=sda
rootpw --plaintext <contraseña>
user --name=bogdan --groups=wheel --plaintext --password=<contraseña>
sshkey --username=bogdan "<clave pública>"
selinux --enforcing
firewall --enabled --service=ssh
firstboot --disable
poweroff

%packages
@^minimal-environment
%end
```

Tiene las mismas opciones que se eligieron en `infra`, cambiando el nombre y la IP de cada máquina: instalación en modo texto, solo en el disco `sda` con particionado automático, red estática, usuario `bogdan` en el grupo `wheel` y el entorno mínimo de paquetes. Además añade tres directivas:

- `sshkey`**:** Mete directamente la clave pública en el `authorized_keys` del usuario `bogdan`, así que no hay que copiarla a mano.
- `selinux` **y** `firewall`**:** Dejan SELinux en modo *enforcing* y el cortafuegos activado, solo con el servicio SSH abierto.
- `poweroff`**:** Apaga la máquina al terminar. Con `reboot`, la máquina volvería a arrancar desde la ISO, encontraría otra vez el disco `OEMDRV` y se reinstalaría en bucle.

Antes de usarlo, la sintaxis se comprobó con `ksvalidator -v RHEL9`, la herramienta oficial de Red Hat (paquete `pykickstart`). El proceso completo tiene tres pasos:

1. **Preparar los discos:** El script `03-kickstart.ps1` genera el `ks.cfg` de cada máquina, crea un disco de 100 MB en FAT32 con la etiqueta `OEMDRV`, copia el fichero dentro y se lo conecta a la máquina. Para formatearlo, Windows tiene que montar el disco un momento (conectarlo como si fuera una unidad más), por eso aparece brevemente en el Explorador. Las contraseñas se piden al ejecutar el script, así que no quedan en el repositorio, y el fichero se escribe con finales de línea LF, que es lo que espera Linux.

2. **Instalar:** Se encienden las tres máquinas a la vez con `Start-VM pg1, pg2, pg3`. La instalación va sola y cada máquina se apaga al terminar.

3. **Quitar el instalador:** El script `04-tras-kickstart.ps1` quita la ISO y el disco `OEMDRV` de las tres máquinas y las enciende ya desde su disco.

La verificación desde Windows confirmó en los tres nodos el nombre y la IP correctos, que solo quedaban los discos `sda` y `sdb`, que se entraba sin contraseña gracias a la clave que instaló Kickstart y que el usuario podía usar `sudo`.

3.9. Actualización y Conexión entre Máquinas

`infra` va a ser el **nodo de control** de Ansible, es decir, la máquina desde la que se configurarán las demás. Ansible no necesita instalar nada en las máquinas que gestiona: se conecta a ellas por SSH y ejecuta las tareas directamente. Para eso `infra` tiene que conocerlas por su nombre y poder entrar en ellas con clave.

Antes de nada se actualizaron las cuatro, que es lo primero que se hace en un servidor recién instalado: la ISO tenía ya unas semanas y desde su publicación habían salido parches de seguridad. En Rocky, el software se instala y se actualiza con `dnf`, el gestor de paquetes, que descarga los paquetes de los repositorios oficiales de la distribución. Si la actualización incluye un kernel nuevo, hay que reiniciar para empezar a usarlo.

```
foreach ($h in 'infra', 'pg1', 'pg2', 'pg3') { ssh -t $h "sudo dnf -y update
    && sudo reboot" }
foreach ($h in 'infra', 'pg1', 'pg2', 'pg3') { ssh -t $h "printf '10.10.10.10
    infra\n10.10.10.11 pg1\n10.10.10.12 pg2\n10.10.10.13 pg3\n' | sudo tee -a
    /etc/hosts" }
```

La opción `-t` de `ssh` hace falta para que `sudo` pueda pedir la contraseña a través de la conexión. El segundo bucle añade las cuatro máquinas al fichero `/etc/hosts` de cada una, que es la agenda local de nombres de Linux: el sistema lo consulta antes que al DNS, así que permite usar `pg1` en lugar de su IP sin necesidad de un servidor DNS propio.

Después, en `infra` se creó su propio par de claves, sin frase de paso (la contraseña opcional que protege la clave privada), porque Ansible tiene que poder usarlas sin que nadie escriba nada. La clave pública se copió a los nodos con `ssh-copy-id`, que hace lo mismo que el comando manual de la sección 3.7, incluidos los permisos y la etiqueta de SELinux:

```
ssh-keygen -t ed25519 -N "" -f ~/.ssh/id_ed25519
for h in pg1 pg2 pg3; do ssh-copy-id $h; done
for h in pg1 pg2 pg3; do ssh $h "hostname; uname -r"; done
```

Los tres nodos respondieron sin pedir contraseña y con el mismo kernel actualizado (`uname -r` muestra su versión, `5.14.0-687.58.1.el9_8.x86_64`). Además, las huellas que mostró `infra` al conectar con los nodos eran las mismas que se vieron antes desde Windows, lo que confirma que en los dos casos se trataba de los mismos servidores.

3.10. Resultado de la Fase

Requisito	Estado
Cuatro máquinas virtuales con Rocky Linux 9.8	Cumplido
Red interna con IP fijas y NAT	Cumplido
Acceso SSH por clave desde Windows a todas las máquinas	Cumplido
Acceso SSH por clave desde `infra` a los nodos de base de datos	Cumplido
Repositorio con su estructura inicial	Cumplido

Tabla 9: Criterio de salida de la Fase 0.

3.11. Diferencias con un Entorno de Producción

Al tratarse de un laboratorio, se han asumido conscientemente las siguientes simplificaciones:

- **Red:** Windows hace de router y de NAT. En producción habría una VLAN dedicada (una red separada lógicamente de las demás dentro de los mismos switches físicos), un router o cortafuegos real y la red de replicación separada de la de los clientes.
- **Almacenamiento:** Las cuatro máquinas comparten un único SSD sin DRAM y usan discos dinámicos. En producción las bases de datos van en almacenamiento dedicado, con discos de tamaño fijo y a menudo con el WAL en un volumen propio.
- **Resolución de nombres:** Se usa `/etc/hosts`. En producción lo resolvería un DNS interno.
- **Contraseñas en Kickstart:** El instalador guarda una copia del Kickstart en `/root/original-ks.cfg` con las contraseñas en claro. En producción irían cifradas (`rootpw -iscrypted`) y ese fichero se borraría tras la instalación.
- **etcd:** Comparte disco con el sistema y con PostgreSQL. En producción va en un disco dedicado de baja latencia, porque si su escritura es lenta, Patroni puede pensar que el primario ha caído y provocar cambios de primario innecesarios.

A. Scripts de Hyper-V

Los scripts se incluyen tal como están en la carpeta `scripts/hyperv/` del repositorio. Se ejecutan en Windows desde PowerShell como administrador y en el orden que indica su número.

A.1. 01-red.ps1

```
<#
.SYNOPSIS
    Crea la red interna del laboratorio en Hyper-V.

.DESCRIPTION
    Switch interno LabSwitch, IP 10.10.10.1/24 para Windows en ese switch
    y NAT para que las VMs (10.10.10.0/24) salgan a Internet.

.NOTES
    Requisitos: PowerShell como Administrador y Hyper-V activado.
    No es idempotente, ejecutar una sola vez. Para repetir, usar antes 01-red-borrar.
    ps1.
```

```
    Windows solo admite un NAT de este tipo: comprobar antes con Get-NetNat.
#>

# Switch interno: solo conecta Windows y las VMs, sin acceso directo a la red física
New-VMSwitch -Name LabSwitch -SwitchType Internal

# Windows hace de router: esta IP es la puerta de enlace de las VMs
New-NetIPAddress -InterfaceAlias "vEthernet (LabSwitch)" -IPAddress 10.10.10.1 -
    PrefixLength 24

# NAT: las VMs salen a Internet pero nadie de fuera puede entrar en ellas
New-NetNat -Name LabNAT -InternalIPInterfaceAddressPrefix 10.10.10.0/24
```

A.2. 01-red-borrar.ps1

```
<#
.SYNOPSIS
    Deshace 01-red.ps1: borra el switch LabSwitch.

.DESCRIPTION
    Borra todos los switches llamados LabSwitch (Hyper-V permite duplicados)
    junto con su tarjeta vEthernet y la IP 10.10.10.1. No toca el Default Switch.

.NOTES
    Requisitos: PowerShell como Administrador y ninguna VM conectada a LabSwitch.
    El NAT no se borra; si hace falta: Remove-NetNat -Name LabNAT
#>

# -Force evita la confirmación por cada switch
Remove-VMSwitch -Name LabSwitch -Force
```

A.3. 02-vms.ps1

```
<#
.SYNOPSIS
    Crea las 4 VMs del laboratorio en Hyper-V.

.DESCRIPTION
    infra, pg1, pg2 y pg3: generación 2, 2 vCPU, RAM fija, disco de sistema de 20 GB
    y disco de datos aparte. Arrancan desde la ISO de Rocky Linux 9.

.NOTES
    Requisitos: PowerShell como Administrador y LabSwitch creado (01-red.ps1).
    No es idempotente: ejecutar una sola vez. Para repetir, borrar antes las VMs.
#>

$iso = 'E:\HyperV\ISO\Rocky-9.8-x86_64-minimal.iso'

foreach ($n in 'infra', 'pg1', 'pg2', 'pg3') {
    # Generación 2 = UEFI y arranque seguro, como un servidor actual
    New-VM $n -Generation 2 -MemoryStartupBytes 3GB -SwitchName LabSwitch -NewVHDPath
    "E:\HyperV\VHD\$n.vhdx" -NewVHDSizeBytes 20GB

    # RAM fija: PostgreSQL se ajusta a la memoria que ve, no debe cambiar en caliente
    Set-VM $n -ProcessorCount 2 -StaticMemory -AutomaticCheckpointsEnabled $false

    # Datos en un disco aparte: si se llena, no tumba el sistema operativo
    New-VHD "E:\HyperV\VHD\$n-datos.vhdx" -SizeBytes 20GB
    Add-VMHardDiskDrive $n -Path "E:\HyperV\VHD\$n-datos.vhdx"
```

```
    # La plantilla por defecto solo arranca Windows; esta acepta Linux
    Add-VMDvdDrive $n -Path $iso
    Set-VMFirmware $n -SecureBootTemplate MicrosoftUEFICertificateAuthority -
    FirstBootDevice (Get-VMDvdDrive $n)
}

# infra aloja más servicios (MinIO, Prometheus...): más RAM y más disco de datos
Set-VM infra -MemoryStartupBytes 4GB
Resize-VHD E:\HyperV\VHD\infra-datos.vhdx -SizeBytes 40GB
```

A.4. 03-kickstart.ps1

```
<#
.SYNOPSIS
    Prepara la instalación automática (Kickstart) de pg1, pg2 y pg3.

.DESCRIPTION
    Para cada VM crea un disco pequeño con la etiqueta OEMDRV que contiene un ks.cfg
    con su nombre e IP, y se lo conecta. Al arrancar desde la ISO, el instalador de
    Rocky encuentra ese ks.cfg solo y hace la instalación sin preguntar nada.
    La VM se apaga sola al terminar.

.NOTES
    Requisitos: PowerShell como Administrador, VMs creadas (02-vms.ps1) y la clave
    $HOME\.ssh\id_ed25519_lab.pub creada.
    Las contraseñas se piden al ejecutar: no se guardan en el repo.
#>

$rootPass = Read-Host "Contraseña de root"
$userPass = Read-Host "Contraseña de bogdan"
$sshKey   = Get-Content "$HOME\.ssh\id_ed25519_lab.pub"

$ips = @{ pg1 = '10.10.10.11'; pg2 = '10.10.10.12'; pg3 = '10.10.10.13' }

foreach ($n in $ips.Keys) {
    # Mismas respuestas que en la instalación manual de infra, cambiando nombre e IP
    $ks = @"
text
lang en_US.UTF-8
keyboard es
timezone Europe/Madrid --utc
network --bootproto=static --device=eth0 --ip=$($ips[$n]) --netmask=255.255.255.0 --
    gateway=10.10.10.1 --nameserver=1.1.1.1,8.8.8.8 --hostname=$n --activate
ignoredisk --only-use=sda
clearpart --all --initlabel --drives=sda
autopart
bootloader --boot-drive=sda
rootpw --plaintext $rootPass
user --name=bogdan --groups=wheel --plaintext --password=$userPass
sshkey --username=bogdan "$sshKey"
selinux --enforcing
firewall --enabled --service=ssh
firstboot --disable
poweroff

%packages
@^minimal-environment
%end
"@
```

```
    # Disco de 100 MB en FAT32 con la etiqueta OEMDRV, que es la que busca el
    instalador
    $vhd = "E:\HyperV\VHD\$n-ks.vhdx"
    $vol = New-VHD $vhd -SizeBytes 100MB | Mount-VHD -Passthru | Initialize-Disk -
    PartitionStyle MBR -Passthru |
           New-Partition -AssignDriveLetter -UseMaximumSize | Format-Volume -
    FileSystem FAT32 -NewFileSystemLabel OEMDRV -Confirm:$false

    # Linux espera finales de línea LF: se quitan los CR de Windows
    [IO.File]::WriteAllText("$($vol.DriveLetter):\ks.cfg", ($ks -replace "`r", ""))

    Dismount-VHD $vhd
    Add-VMHardDiskDrive $n -Path $vhd
}
```

A.5. 04-tras-kickstart.ps1

```
<#
.SYNOPSIS
    Deja pg1, pg2 y pg3 listas tras la instalación con Kickstart.

.DESCRIPTION
    Saca la ISO del lector, desconecta el disco OEMDRV con el ks.cfg y enciende las
    VMs,
    que ya arrancan desde su disco.

.NOTES
    Requisitos: PowerShell como Administrador y las 3 VMs apagadas tras instalar.
    Los ficheros *-ks.vhdx quedan en E:\HyperV\VHD; se pueden borrar a mano.
#>

foreach ($n in 'pg1', 'pg2', 'pg3') {
    Set-VMDvdDrive -VMName $n -Path $null
    Get-VMHardDiskDrive -VMName $n | Where-Object Path -like '*-ks.vhdx' | Remove-
    VMHardDiskDrive
    Start-VM $n
}
```

B. Registro de Decisiones

Las decisiones de diseño se documentan en la carpeta `docs/decisiones/` con el formato ADR (*Architecture Decision Record*): contexto, decisión, alternativas y consecuencias. Este apéndice resume las que están en vigor.

B.1. ADR 0001: Hipervisor Hyper-V

- **Contexto:** Equipo con Windows 11 Pro, i5-11400H, 32 GB de RAM y 484 GB libres en E:. Hacen falta cuatro máquinas con un rendimiento de disco realista para PostgreSQL.
- **Decisión:** Usar Hyper-V, con las máquinas y sus discos en `E:\HyperV`.
- **Alternativas:** VirtualBox, que sobre la capa de Hyper-V (WSL2, integridad de memoria) pierde rendimiento de disco, y Proxmox, que necesita una máquina dedicada.
- **Consecuencias:** La creación de las máquinas se automatiza con PowerShell. La memoria dinámica se desactiva para que PostgreSQL trabaje con RAM fija.

B.2. ADR 0002: Sistema Operativo Rocky Linux 9

- **Contexto:** El laboratorio debe parecerse a un entorno de producción de gran empresa o consultora, donde predomina la familia RHEL. El planteamiento inicial era Debian 12, que ya está al final de su soporte normal.
- **Decisión:** Rocky Linux 9 en las cuatro máquinas, con SELinux activado y firewalld, y PostgreSQL 16 desde los repositorios oficiales PGDG.
- **Alternativas:** Debian 12 o 13, más sencilla pero menos representativa del entorno empresarial; Ubuntu LTS, habitual en startups y en la nube; Rocky Linux 10, más reciente pero todavía menos extendida.
- **Consecuencias:** Los paquetes se instalan con `dnf` y los datos de PostgreSQL van en `/var/lib/pgsql/16/data`. Hay que gestionar SELinux y firewalld, y SELinux no se desactiva para resolver problemas.